WEEK-1

OBJECTIVE :

Demonstration of Fork() System Call

Code:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;

    printf("Before fork()\n");

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0) {
        printf("This is the child process = %d\n", getpid());
    }
    else {
        printf("This is the parent process=%d\n", getpid());
    }


    return 0;
}
```

Output:

```
Before fork()
This is the parent process = 17624
This is the child process = 17625
```

WEEK-2

OBJECTIVE:

Parent process computes the sum of even numbers and child process computes the sum of odd numbers using fork().

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int i, sum_even = 0, sum_odd = 0;
    int n = 10;

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0) {
        for (i = 1; i <= n; i += 2) {
            sum_odd += i;
        }
        printf("Child process (PID=%d) - Sum of odd numbers = %d\n", getpid(), sum_odd);
    }
    else {
        for (i = 2; i <= n; i += 2) {
            sum_even += i;
        }
        wait(NULL);
        printf("Parent process (PID=%d) - Sum of even numbers = %d\n", getpid(), sum_even);
    }

    return 0;
}
```

Output:

```
Child process (PID=14645) - Sum of odd numbers = 25
Parent process (PID=14644) - Sum of even numbers = 30
```

WEEK-3

OBJECTIVE:

Demonstration of wait() system call.

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid == 0) {
        printf("Child process is running\n");
    }
    else {
        wait(NULL);
        printf("Parent process resumes after child finishes\n");
    }

    return 0;
}
```

Output:

```
Child process is running
Parent process resumes after child finishes
```

WEEK-4

OBJECTIVE:

Implementation of Orphan Process and Zombie Process.

Code:

Orphan Process:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
   pid_t pid = fork();

   if (pid < 0) {
      perror("fork failed");
      return 1;
   }
   else if (pid == 0) {
      sleep(5);
      printf("Child process (PID=%d) running as orphan\n", getpid());
      exit(0);
   }
   else {
      printf("Parent process (PID=%d) exiting\n", getpid());
      exit(0);
   }

   return 0;
}
```

Output:

```
Parent process (PID=9749) exiting
```

Zombie Process:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0) {
        printf("Child process (PID=%d) exiting immediately\n", getpid());
        exit(0);
    }
    else {
        sleep(10);
        printf("Parent process (PID=%d) finished sleeping\n", getpid());
    }

    return 0;
}
```

Output:

```
Child process (PID=4345) exiting immediately
```

WEEK - 05

Objective :

First Come First Served Scheduling Algorithm

Code :

```
#include <stdio.h>
int main() {
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int burst_time[n], waiting_time[n], turnaround_time[n];
    int total_wt = 0, total_tat = 0;
    for (i = 0; i < n; i++) {
        printf("Enter burst time for process %d: ", i + 1);
        scanf("%d", &burst_time[i]);
    }
    waiting_time[0] = 0;
    for (i = 1; i < n; i++) {
        waiting_time[i] = waiting_time[i - 1] + burst_time[i - 1];
    }
    for (i = 0; i < n; i++) {
        turnaround_time[i] = burst_time[i] + waiting_time[i];
    }
    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for (i = 0; i < n; i++) {
        printf("%d\t%d\t\t%d\t\t%d\n", i + 1, burst_time[i], waiting_time[i], turnaround_time[i]);
        total_wt += waiting_time[i];
        total_tat += turnaround_time[i];
    }
    printf("\nAverage Waiting Time = %.2f\n", (float)total_wt / n);
    printf("Average Turnaround Time = %.2f\n", (float)total_tat / n);
    return 0;
}
```

Output:

Output Clear

```
Enter number of processes: 4
Enter burst time for process 1: 5
Enter burst time for process 2: 3
Enter burst time for process 3: 8
Enter burst time for process 4: 6

Process    Burst Time   Waiting Time    Turnaround Time
  1         5      0           5
  2         3      5           8
  3         8      8           16
  4         6      16          22

Average Waiting Time = 7.250000
Average Turnaround Time = 12.750000


=== Code Execution Successful ===
```

WEEK - 06

Objective :

Shortest Job First Scheduling Algorithm

Code For Non-Preemptive:

```
#include <stdio.h>
int main() {
    int n, i, j;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int arrival_time[n], burst_time[n], temp_burst[n], waiting_time[n], turnaround_time[n];
    int completed[n];
    int current_time = 0, completed_count = 0;
    for (i = 0; i < n; i++) {
        printf("Enter arrival time for process %d: ", i + 1);
        scanf("%d", &arrival_time[i]);
        printf("Enter burst time for process %d: ", i + 1);
        scanf("%d", &burst_time[i]);
        temp_burst[i] = burst_time[i];
        completed[i] = 0;  // not completed yet
    }
    while (completed_count < n) {
        int idx = -1;
        int min_burst = 100000;
        for (i = 0; i < n; i++) {
            if (arrival_time[i] <= current_time && completed[i] == 0) {
                if (burst_time[i] < min_burst) {
                    min_burst = burst_time[i];
                    idx = i;
                }
                // if burst times are equal, choose process with earlier arrival time
                else if (burst_time[i] == min_burst) {
                    if (arrival_time[i] < arrival_time[idx]) {
                        idx = i;
                    }
```

```
            }
        }
    }
    if (idx != -1) {
        waiting_time[idx] = current_time - arrival_time[idx];
        if (waiting_time[idx] < 0) waiting_time[idx] = 0;
        current_time += burst_time[idx];
        turnaround_time[idx] = current_time - arrival_time[idx];
        completed[idx] = 1;
        completed_count++;
    } else {

        current_time++;
    }
  }
  float total_wt = 0, total_tat = 0;
  printf("\nProcess\tArrival Time\tBurst Time\tWaiting Time\tTurnaround Time\n");
  for (i = 0; i < n; i++) {
    printf("%d\t%d\t\t%d\t\t%d\t\t%d\n", i + 1, arrival_time[i], temp_burst[i], waiting_time[i],
turnaround_time[i]);
    total_wt += waiting_time[i];
    total_tat += turnaround_time[i];
  }
  printf("\nAverage Waiting Time = %.2f\n", total_wt / n);
  printf("Average Turnaround Time = %.2f\n", total_tat / n);

  return 0;
}
```

Output:

```
Output                                                    Clear

Enter number of processes: 4
Enter arrival time for process 1: 20
Enter burst time for process 1: 40
Enter arrival time for process 2: 30
Enter burst time for process 2: 50
Enter arrival time for process 3: 36
Enter burst time for process 3: 78
Enter arrival time for process 4: 45
Enter burst time for process 4: 87

Process Arrival Time    Burst Time  Waiting Time    Turnaround Time
1   20      40      0       40
2   30      50      30      80
3   36      78      74      152
4   45      87      143     230

Average Waiting Time = 61.75
Average Turnaround Time = 125.50


=== Code Execution Successful ===
```

Code For Preemptive:

```
#include <stdio.h>
int main() {
    int n, i, completed = 0, current_time = 0, min_index;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int arrival_time[n], burst_time[n], remaining_time[n], waiting_time[n], turnaround_time[n];
    int is_completed[n];
    for (i = 0; i < n; i++) {
        printf("Enter arrival time for process %d: ", i + 1);
        scanf("%d", &arrival_time[i]);
        printf("Enter burst time for process %d: ", i + 1);
        scanf("%d", &burst_time[i]);
        remaining_time[i] = burst_time[i];
        is_completed[i] = 0;
        waiting_time[i] = 0;
```

```
    }
    while (completed != n) {
        min_index = -1;
        int min_remaining = 100000;
        for (i = 0; i < n; i++) {
            if (arrival_time[i] <= current_time && !is_completed[i] && remaining_time[i] <
min_remaining && remaining_time[i] > 0) {
                min_remaining = remaining_time[i];
                min_index = i;
            }
        }
        if (min_index == -1) {
            current_time++;
            continue;
        }
        remaining_time[min_index]--;
        current_time++;

        if (remaining_time[min_index] == 0) {
            is_completed[min_index] = 1;
            completed++;
            turnaround_time[min_index] = current_time - arrival_time[min_index];
            waiting_time[min_index] = turnaround_time[min_index] - burst_time[min_index];
            if (waiting_time[min_index] < 0) waiting_time[min_index] = 0;
        }
    }
    float total_wt = 0, total_tat = 0;
    printf("\nProcess\tArrival Time\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for (i = 0; i < n; i++) {
        printf("%d\t%d\t\t%d\t\t%d\t\t%d\n", i + 1, arrival_time[i], burst_time[i], waiting_time[i],
turnaround_time[i]);
        total_wt += waiting_time[i];
        total_tat += turnaround_time[i];
    }
    printf("\nAverage Waiting Time = %.2f\n", total_wt / n);
```

```
    printf("Average Turnaround Time = %.2f\n", total_tat / n);
    return 0;
}
```

Output:

Output Clear

```
Enter number of processes: 3
Enter arrival time for process 1: 20
Enter burst time for process 1: 40
Enter arrival time for process 2: 30
Enter burst time for process 2: 50
Enter arrival time for process 3: 45
Enter burst time for process 3: 65

Process Arrival Time    Burst Time  Waiting Time    Turnaround Time
1   20      40      0       40
2   30      50      30      80
3   45      65      65      130

Average Waiting Time = 31.67
Average Turnaround Time = 83.33


=== Code Execution Successful ===
```

WEEK - 07

Objective:

Priority scheduling algorithm

```
#include <stdio.h>
#include <limits.h>

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], pr[n];
    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("Enter AT BT Priority for P%d: ", pid[i]);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
    }

    // Non-Preemptive
    int rt[n], tat[n], done[n], t = 0, c = 0;
    float twt = 0, ttat = 0;
    for (int i = 0; i < n; i++) done[i] = 0;

    printf("\nNon-Preemptive:\n");
    while (c < n) {
        int idx = -1, minP = INT_MAX;
        for (int i = 0; i < n; i++) {
            if (!done[i] && at[i] <= t && pr[i] < minP) {
                minP = pr[i];
                idx = i;
            }
        }
        if (idx == -1) { t++; continue; }
```

```
    rt[idx] = t - at[idx];
    tat[idx] = rt[idx] + bt[idx];
    t += bt[idx];
    done[idx] = 1;
    c++;

    twt += rt[idx];
    ttat += tat[idx];
    printf("P%d: RT=%d TAT=%d\n", pid[idx], rt[idx], tat[idx]);
}
printf("AWT=%.2f ATAT=%.2f\n", twt/n, ttat/n);

// Preemptive
int rem[n], first[n];
t = 0; c = 0; twt = 0; ttat = 0;
for (int i = 0; i < n; i++) { rem[i] = bt[i]; first[i] = -1; }

printf("\nPreemptive:\n");
while (c < n) {
    int idx = -1, minP = INT_MAX;
    for (int i = 0; i < n; i++) {
        if (at[i] <= t && rem[i] > 0 && pr[i] < minP) {
            minP = pr[i];
            idx = i;
        }
    }
    if (idx == -1) { t++; continue; }

    if (first[idx] == -1) first[idx] = t;
    rem[idx]--; t++;

    if (rem[idx] == 0) {
        tat[idx] = t - at[idx];
        rt[idx] = first[idx] - at[idx];
```

```
            c++;
            twt += rt[idx];
            ttat += tat[idx];
            printf("P%d: RT=%d TAT=%d\n", pid[idx], rt[idx], tat[idx]);
        }
    }
    printf("AWT=%.2f ATAT=%.2f\n", twt/n, ttat/n);

    return 0;
}
```

Output:

Output Clear

```
Enter number of processes: 4
Process 1 arrival time: 35
Process 1 burst time: 56
Process 1 priority (lower value = higher priority): 45
Process 2 arrival time: 20
Process 2 burst time: 76
Process 2 priority (lower value = higher priority): 49
Process 3 arrival time: 38
Process 3 burst time: 59
Process 3 priority (lower value = higher priority): 76
Process 4 arrival time: 94
Process 4 burst time: 89
Process 4 priority (lower value = higher priority): 48

P#  Arrival Burst   Priority    Waiting Turnaround
1   35  56  45      0   56
2   20  76  49      145 221
3   38  59  76      203 262
4   94  89  48      0   89

Average Waiting Time: 87.00
Average Turnaround Time: 157.00
```

WEEK – 08

Objective:

Round Robin Scheduling algorithm

```
#include <stdio.h>
void findWaitingTime(int n, int bt[], int at[], int wt[], int quantum)
{
	int rem_bt[n];
	for (int i = 0; i < n; i++)
	rem_bt[i] = bt[i];
	int t = 0;
	int done;
	while (1) {
		done = 1;
		for (int i = 0; i < n; i++) {
			if (at[i] <= t && rem_bt[i] > 0) {
				done = 0;
				if (rem_bt[i] > quantum) {
					t += quantum;
					rem_bt[i] -= quantum;
				}
				else {
					t += rem_bt[i];
					wt[i] = t - bt[i] - at[i];
					rem_bt[i] = 0;
				}
			}
		}
		if (done == 1)
			break;
	}
}
```

```
void findTurnAroundTime(int n, int bt[], int wt[], int tat[]) {
	for (int i = 0; i < n; i++)
	tat[i] = bt[i] + wt[i];
}
void findavgTime(int n, int bt[], int at[], int quantum) {
	int wt[n], tat[n];
	int total_wt = 0, total_tat = 0;
	findWaitingTime(n, bt, at, wt, quantum);
	findTurnAroundTime(n, bt, wt, tat);
	printf("\nPriority\tArrival_time\tBurst_time\tWaiting_time\tTurn_around_time\n");
	for (int i = 0; i < n; i++) {
		total_wt += wt[i];
		total_tat += tat[i];
		printf(" %d\t\t %d\t\t %d\t\t %d\t %d\n", i + 1, at[i], bt[i], wt[i], tat[i]);
	}
	printf("\nAverage waiting time = %.2f", (float)total_wt / (float)n);
	printf("\nAverage turn around time = %.2f\n", (float)total_tat / (float)n);
}
int main() {
	int n, i, quantum;
	printf("Enter number of processes: ");
	scanf("%d", &n);
	int arrival_time[n], burst_time[n], priority[n];
	for (i = 0; i < n; i++) {
		printf("Process %d arrival time: ", i + 1);
		scanf("%d", &arrival_time[i]);
		printf("Process %d burst time: ", i + 1);
		scanf("%d", &burst_time[i]);
		printf("Process %d priority (lower value = higher priority): ", i + 1);
		scanf("%d", &priority[i]);
	}
```

```
    printf("Enter Time Quantum: ");
    scanf("%d", &quantum);
    findavgTime(n, burst_time, arrival_time, quantum);
    return 0;
}
```

Output :

Output Clear

```
Enter number of processes: 4
Process 1 arrival time: 0
Process 1 burst time: 5
Process 1 priority (lower value = higher priority): 1
Process 2 arrival time: 1
Process 2 burst time: 4
Process 2 priority (lower value = higher priority): 2
Process 3 arrival time: 2
Process 3 burst time: 6
Process 3 priority (lower value = higher priority): 3
Process 4 arrival time: 3
Process 4 burst time: 3
Process 4 priority (lower value = higher priority): 1
Enter Time Quantum: 2

Priority    Arrival_time    Burst_time  Waiting_time    Turn_around_time
 1       0       5       11  16
 2       1       4       7   11
 3       2       6       10  16
 4       3       3       9   12

Average waiting time = 9.25
Average turn around time = 13.75


=== Code Execution Successful ===
```

WEEK – 09

Objective:

Implementation of Named Pipe

```
// Creating fifo/named pipe  (1.c)
#include<stdio.h>
#include<sys/types.h>
#include<sys/stat.h>        int main()
    {
        int res;
        res = mkfifo("fifo1",0777); //creates a named pipe with the name fifo1
printf("named pipe created\n");
    }

//   Writing to a fifo/named pipe (2.c)
#include<unistd.h>
#include<stdio.h>
#include<fcntl.h> int main()
{
        int res,n;
        res=open("fifo1",O_WRONLY); write(res,"Message",7);
        printf("Sender Process %d sent the data\n",getpid());
}

// Reading from the named pipe ( 3.c )
#include<unistd.h>
#include<stdio.h>
#include<fcntl.h>
```

```
int main()
{
        int res,n; char buffer[100]; res=open("fifo1",O_RDONLY);
        n=read(res,buffer,100); printf("Reader process %d started\n",getpid());
        printf("Data received by receiver %d is: %s\n",getpid(), buffer);
}
```

Output :